

# Offline-First Flutter Applications

Architectural Patterns, Synchronization Strategies, and  
Technical Trade-offs



# Executive Summary

---

## The Paradigm Shift

Modern mobile development has shifted from an "online-first" assumption to an **"offline-first" requirement**. An offline-first architecture is not a feature; it is a fundamental structural commitment that inverts the traditional client-server relationship. The local device becomes the **primary authority** for the UI, while the network serves merely as a background synchronization utility.

## The Core Dichotomy

- > **Firebase Ecosystem:** Offers rapid development with a cohesive but opaque synchronization engine. Best for prototyping and simple data models.
- > **Custom SQLite (Enterprise):** Provides granular control over data consistency, query performance, and schema evolution. Best for complex, mission-critical applications.

# Theoretical Framework: The Inversion of Truth

## Traditional Model

### Request-Response Cycle

In traditional apps, the UI waits for the server. A user action triggers a network request, and the UI updates only upon a successful response.

$UI = f(Network\_Response)$

 **Fragile: Latency breaks UX.**

## Offline-First

### Inversion of Truth

**Local Read Authority:** UI reads exclusively from local persistence, guaranteeing consistent 60fps rendering.

**Optimistic Write Authority:** User actions mutate local state immediately. Network requests are background side-effects.

$UI = f(Local\_Database)$

# The CAP Theorem in Mobile Context

Mobile devices operate in a perpetually partitioned state. Therefore, offline-first apps must prioritize **Availability** and **Partition Tolerance** over immediate Consistency.



## Partition Tolerance

The system continues to operate despite an arbitrary number of messages being dropped or delayed by the network.



## Availability

Every request receives a (non-error) response, without the guarantee that it contains the most recent write.



## Consistency

Sacrificed. We accept **Eventual Consistency**, creating the architectural challenge of synchronization and conflict resolution.

# Data Synchronization Topologies

| Topology             | Mechanism                                          | Latency   | Battery Impact        | Ideal Use Case                             |
|----------------------|----------------------------------------------------|-----------|-----------------------|--------------------------------------------|
| Push-Based           | WebSockets (e.g., Firebase, GraphQL Subscriptions) | Real-time | High (Keep-alive)     | Chat, Live Tracking, Collaborative Editing |
| Pull-Based (Polling) | Fixed Interval Requests (REST GET)                 | High      | High (Radio wake-ups) | Non-critical updates, Legacy backends      |
| Hybrid (Delta Sync)  | Request "changes since X" on trigger               | Moderate  | Low                   | Enterprise Data, Inventory, Social Feeds   |

*The choice between Firebase and SQLite is largely a choice between a managed **Push-Based** topology and a developer-constructed **Hybrid** topology.*

# Approach A: The Firebase Ecosystem

Google's Firebase offers an "out-of-the-box" solution by abstracting local storage and synchronization.

## The Local Cache Mechanism

- > **Interceptor Pattern:** The SDK intercepts queries and inspects the local persistence layer first.
- > **Pending Write Queue:** Writes are applied atomically to the local cache and appended to a persistent queue for background syncing.
- > **Latency Compensation:** The UI reflects changes instantly via snapshots, bypassing the network stack entirely.





# Firestore Implementation Details

---

## Handling Data Availability

A robust implementation must explicitly handle scenarios where the network is unavailable and fallback to the cache. Standard error handling often misses this nuance.

## Configuration

While persistence is enabled by default, cache size management is critical for enterprise apps to prevent the eviction of vital operational data.

# Critical Limitations & Risks

## Indexing Bottleneck

Firestore guarantees performance via server-side indexing. However, the **local cache lacks these indexes**. Offline queries often result in full table scans, causing significant UI jank with large datasets.

## Billing Anomalies

Syncing after a long offline period can trigger thousands of read operations instantly (one per updated document), leading to unexpected "bill shock."

## Transactional Integrity

Transactions strictly require online connectivity. Complex business logic involving prerequisite checks cannot be executed atomically while offline.

## Opaque Cache

The cache is a "Black Box" managed by LRU (Least Recently Used). Developers cannot "pin" critical documents, risking data eviction before use.



# Approach B: Custom SQLite (Enterprise Standard)

For applications requiring strict data sovereignty or high-performance offline querying, a custom architecture using **Drift** (formerly Moor) is the superior choice.

| Feature     | Sqflite (Raw)           | Drift (Recommended)        |
|-------------|-------------------------|----------------------------|
| Type Safety | None (Dynamic Maps)     | Full Compile-time Safety   |
| Reactivity  | Manual StreamController | Built-in Streams (watch()) |
| Boilerplate | High (Manual Mapping)   | Low (Code Generation)      |
| Migrations  | Manual SQL Scripts      | Managed Schema Versioning  |

# Schema Design: The "Tombstone" Pattern

---

## Identification Strategy

Auto-increment IDs cause collisions in distributed systems. **UUIDs** generated on the client are mandatory.

## Sync Metadata

Every table requires tracking columns to manage the sync state:

- > **updatedAt:** For Last-Write-Wins resolution.
- > **isDirty:** Flags records needing sync.
- > **deletedAt:** The "Tombstone". We never physically delete rows; we mark them to ensure the deletion propagates to the server.

# The Sync Engine: Push Loop

## The Outbox Pattern

We do not call the API immediately upon user action. Instead, we insert a record into a local SyncQueue table.

- 1. Capture Intent:** User saves data -> Insert into DB + Insert into SyncQueue (Atomic Transaction).
- 2. Process Queue:** Background worker reads FIFO queue.
- 3. Execute:** Attempt API call.
- 4. Finalize:** If 200 OK -> Delete Queue Item + Clear isDirty. If Fail -> Retry later.



# The Sync Engine: Pull Loop (Delta Sync)

## The Problem

Downloading the entire database on every sync is bandwidth-prohibitive and slow.

## The Solution: Delta Sync

The client requests only what has changed since the last successful synchronization checkpoint.

```
GET /api/sync?since=2023-10-27T10:00:00Z
```

## Logic Flow

1. **Request:** Client sends last\_synced\_at timestamp.
2. **Query:** Server finds records where updated\_at > timestamp.
3. **Upsert:** Client iterates response:
  - > If deleted\_at is set -> Local Delete.
  - > Else -> Insert or Update.
4. **Checkpoint:** Update local last\_synced\_at to server time.

# Background Execution with Workmanager

---

To ensure eventual consistency, synchronization must occur even when the app is closed. The workmanager package bridges the gap to Android WorkManager and iOS BGTaskScheduler.



## Android

Reliable. Periodic tasks can be scheduled with a minimum interval of 15 minutes.



## iOS Constraints

Highly restrictive. The OS decides when to run tasks based on user behavior (heuristics). Execution is **not guaranteed**.

---

**Best Practice:** Use Workmanager for "best effort" background sync, but *always* trigger an immediate sync when the app enters the foreground (didChangeAppLifecycleState).

# Conflict Resolution Strategies



## Last Write Wins (LWW)

**Logic:** if (remote.time > local.time) overwrite.

**Pros:** Simple to implement.

**Cons:** Data loss. If User A changes Title and User B changes Body, one entire record is lost.



## Semantic Merge

**Logic:** Field-level inspection.  
Apply changes only to modified fields.

**Pros:** Preserves data integrity.

**Cons:** Requires custom business logic per entity.



## CRDTs

**Logic:** Mathematical convergence (Conflict-free Replicated Data Types).

**Pros:** Mathematically

guaranteed consistency.

**Cons:** High storage overhead (operation history) and complexity.

# Architecture Comparison

| Feature         | Firebase (Firestore)    | Custom SQLite (Drift)   | Implication                                            |
|-----------------|-------------------------|-------------------------|--------------------------------------------------------|
| Data Topology   | NoSQL (Graph)           | Relational (SQL)        | Drift allows rigorous normalization.                   |
| Offline Queries | Restricted (Full Scans) | Unrestricted (Indexed)  | Firestore is unsuitable for complex offline reporting. |
| Transactions    | Online Only             | Offline Supported       | Critical for financial/inventory integrity.            |
| Search          | None Offline            | FTS5 (Full Text Search) | SQLite allows "Search as you type" offline.            |
| Cost Model      | Usage (Reads/Writes)    | Fixed (Infrastructure)  | Firebase costs can explode with inefficient sync.      |



# Recommendations

## Choose Firebase When:

- > **Prototyping:** Speed is the primary metric.
- > **Social Apps:** Simple data relationships; LWW is acceptable.
- > **Live Collaboration:** Real-time push is a core feature.

## Choose Custom SQLite When:

- > **Enterprise Field Apps:** Offline for days; complex filtering needed.
- > **Transaction-Critical:** Cannot risk data corruption.
- > **Cost-Sensitive Scaling:** High read volume prohibits per-read billing.

**The "Third Way":** Middleware solutions like **PowerSync** offer a middle ground, providing SQLite performance with managed sync infrastructure.

# Synchronization

Conflict-Free Replicated Data Types (CRDTs) and Lattice Theory

# Executive Summary

---

## The Challenge

Distributed systems face a binary choice between strong consistency and availability (CAP theorem). Traditional consensus algorithms (Paxos, Raft) sacrifice availability during partitions, which is unacceptable for real-time, collaborative applications.

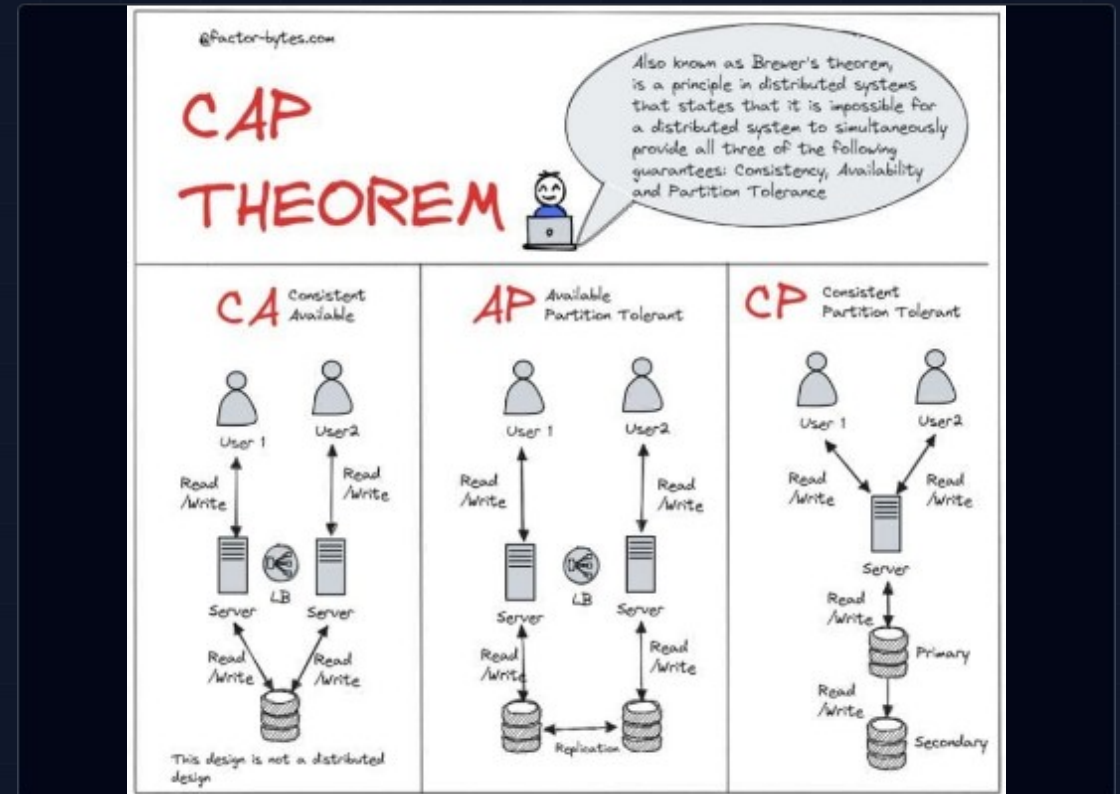
## The CRDT Solution

Conflict-free Replicated Data Types offer a rigorous alternative rooted in Abstract Algebra. By structuring data as monotonic join-semilattices, CRDTs guarantee Strong Eventual Consistency (SEC) without runtime arbitration.

# The Consistency-Availability Trade-off

The CAP theorem posits that a distributed store can only provide two of three guarantees: Consistency, Availability, and Partition Tolerance.

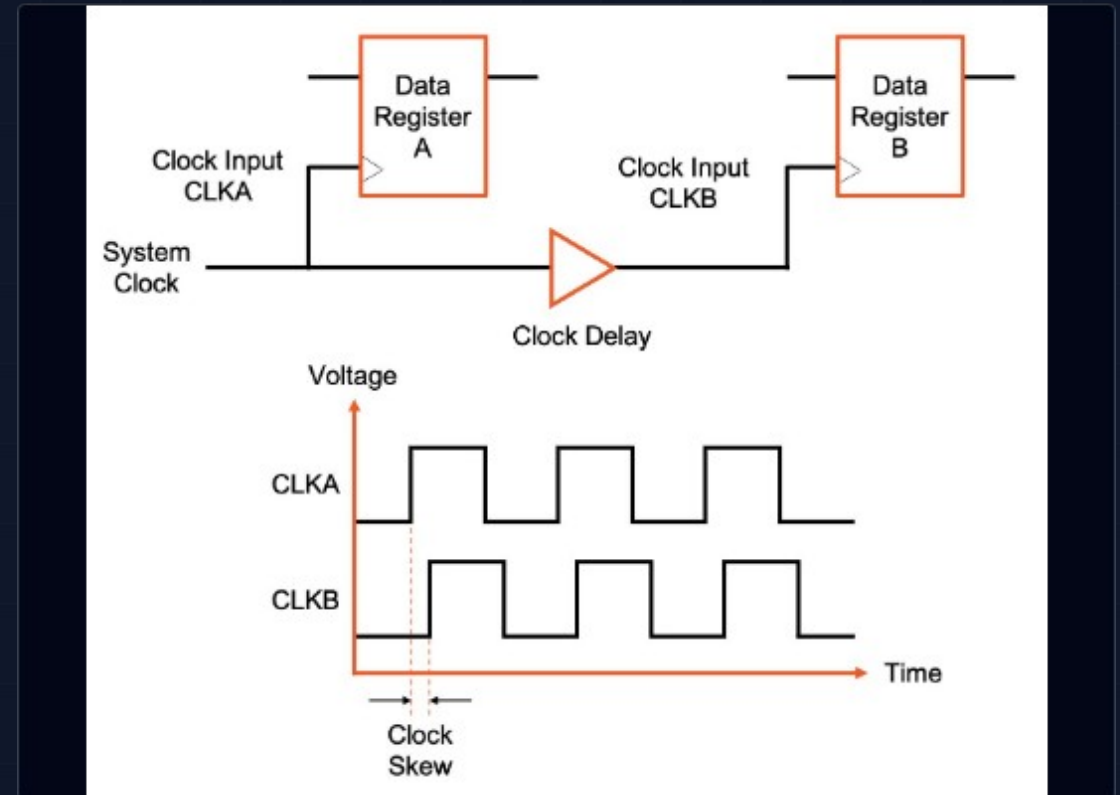
- CP Systems: Refuse writes during partitions to maintain consistency (e.g., Spanner, Etcd). High latency.
- AP Systems: Prioritize uptime and low latency, allowing replicas to diverge. The challenge is reconciliation.



# The Failure of "Last-Write-Wins"

Naive eventual consistency often relies on "Last-Write-Wins" (LWW) based on wall-clock timestamps. This approach is mathematically unsound in asynchronous environments.

- Clock Skew: "Now" is a local illusion. A physically later write can be overwritten by a lagging clock.
- Data Loss: LWW discards information rather than merging it. Concurrent edits to different fields may obliterate each other.



# Mathematical Foundations: Order Theory

## Partial Orders

For a state set  $S$ , we define a relation satisfying:

- Reflexivity:  $x \leq x$
- Antisymmetry:  $x \leq y \wedge y \leq x \Rightarrow x = y$
- Transitivity:  $x \leq y \wedge y \leq z \Rightarrow x \leq z$

## Join-Semilattices

To resolve concurrent states, we need a Least Upper Bound (LUB), or supremum. A Join-Semilattice is a set where every pair has a unique LUB.  $x, y \quad m = x \sqcup y$

This represents the union of all information in both states.

# The Lattice of Convergence

In a lattice-based system, the state only moves "upwards" towards a more complete state. Convergence is not arbitration, but a calculation.

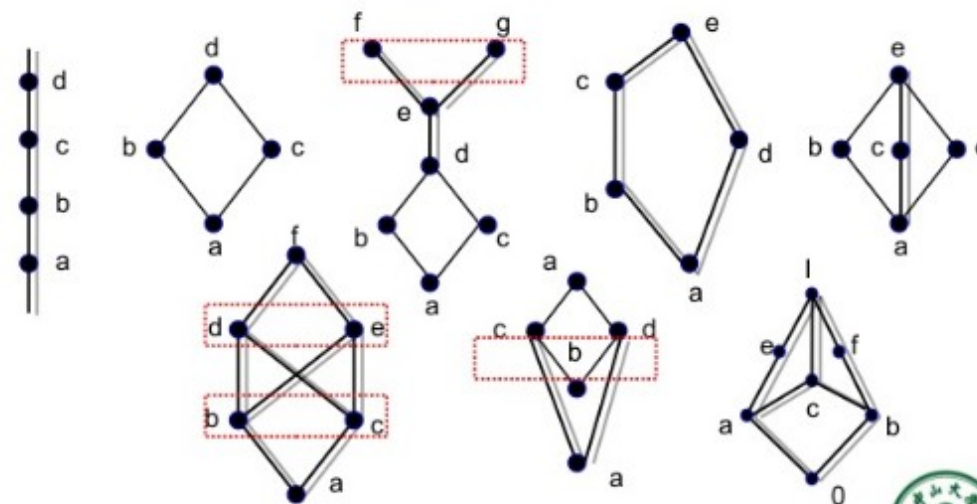
If Replica A has state  $S_A$  and Replica B has  $S_B$ , the merged state is:

$$S_{\text{merged}} = S_A \sqcup S_B$$

This guarantees that any two replicas that have seen the same updates will be in the exact same state.

- Example 4

Which of the Hasse diagrams represent lattices?



# The ACI Properties

---



## Associativity

$$(x \sqcup y) \sqcup z = x \sqcup (y \sqcup z)$$

Grouping doesn't matter. Enables arbitrary gossip topologies.



## Commutativity

$$x \sqcup y = y \sqcup x$$

Order doesn't matter.  
Messages can arrive in any sequence.



## Idempotence

$$x \sqcup x = x$$

Duplication is fine. Processing the same message twice is a no-op.



# Taxonomy: State-based vs. Op-based

---

## State-based (CvRDT)

Mechanism: Replicas broadcast their full local state.

Protocol: Gossip / Anti-Entropy.

Pros: Robust against network failures; simple messaging requirements.

Cons: High bandwidth usage (sending whole state for small changes).

## Operation-based (CmRDT)

Mechanism: Replicas broadcast individual operations (e.g., "insert 'a'").

Protocol: Reliable Broadcast / Pub-Sub.

Pros: Highly bandwidth efficient.

Cons: Requires strict causal delivery guarantees; fragile if ops are lost.

# Primitives: The Algebra of Counting

## G-Counter (Grow-Only)

A simple integer fails merge ( ). A G-Counter uses a vector of integers, one per replica.

Merge: Pointwise Maximum.

$\text{merged}[r] = \max(\text{local}[r], \text{remote}[r])$   
 $\max(1, 1) = 1$

## PN-Counter (Pos-Neg)

To support decrements, we compose two G-Counters: (increments) and (decrements).

Value:  
 $\text{Value}(P) - \text{Value}(N)$

The state always grows monotonically, even if the value fluctuates.

# Set Theory: The OR-Set

---



Unique Tags: Every "add" operation generates a unique tag (dot). Adding "milk" twice creates two distinct entries: ("milk", A1) and ("milk", A2).



Observed-Remove: When a user removes "milk", the system identifies all observed tags and moves them to a "Removed" set (tombstones).



Add-Wins Strategy: If Replica A adds "milk" concurrently with Replica B removing it, the new add survives because B could not have "observed" A's new tag.

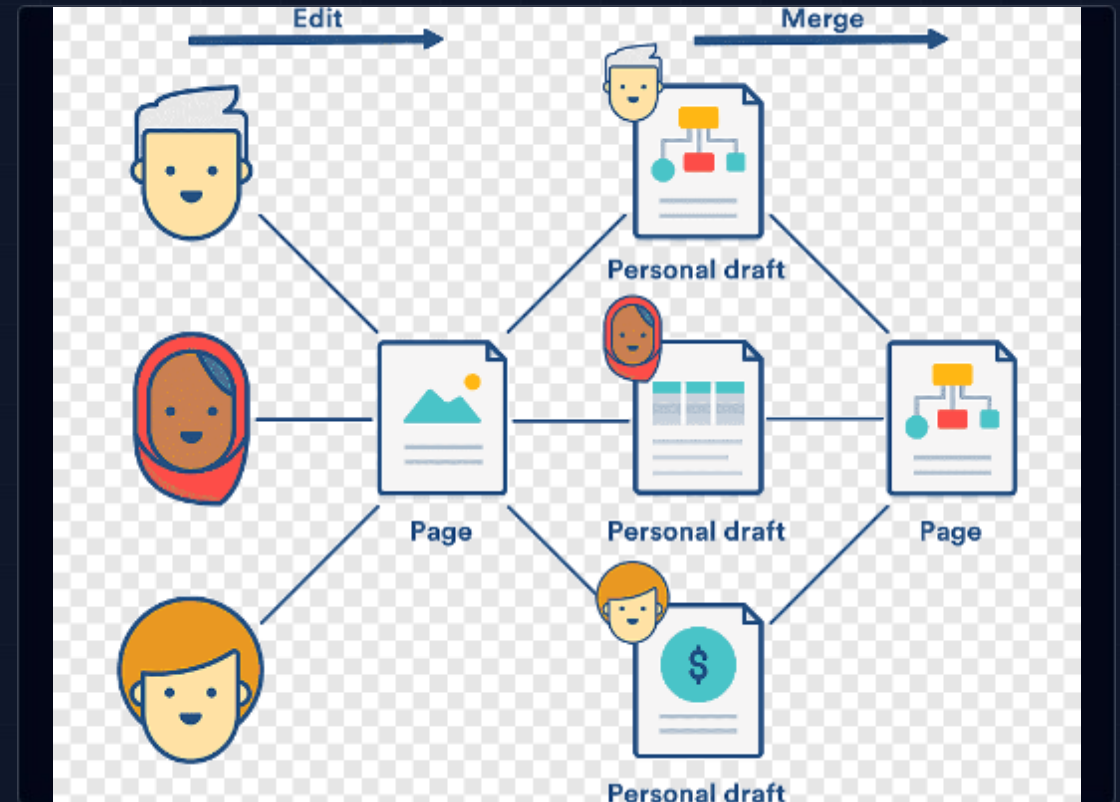


Lattice Definition:  $S_{merged} = (S_A \cup S_B) \setminus (T_A \cup T_B)$

# Sequence CRDTs: Text Editing

Collaborative editing requires handling concurrent inserts at arbitrary positions. Naive indexing fails (Index 5 shifts to 6).

- The Indexing Problem: Fixed indices cannot track dynamic content.
- RGA (Replicated Growable Array): Uses a linked-list structure where every character has a unique timestamp ID.
- Tombstones: Deleted characters are marked "dead" but kept to maintain relative structure for concurrent edits.



# Optimization: Delta-CRDTs

---

## The Bandwidth Problem

Standard CvRDTs must transmit their entire state to merge. For a large G-Set with a million items, adding a single item triggers a massive transfer.

## The Delta Solution

Delta-CRDTs define a mutator that returns a small state fragment, or  $\delta$  "delta" ( ).

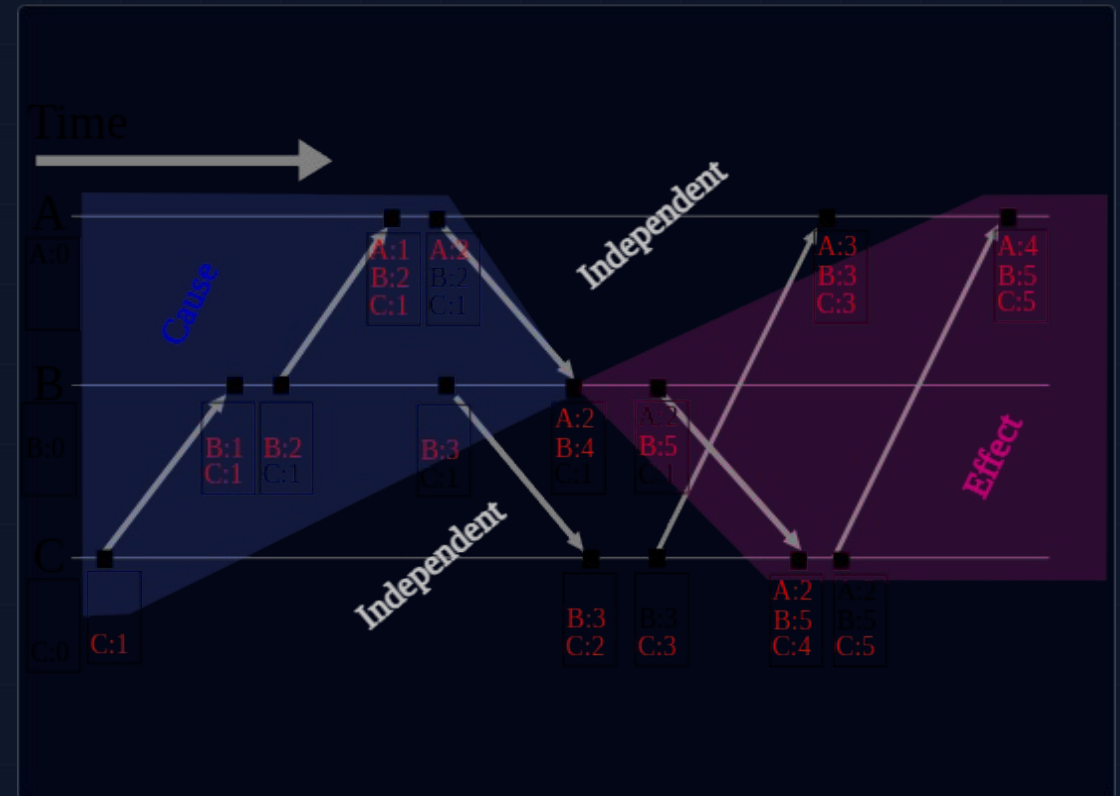
$$S_{\text{new}} = S \sqcup \delta$$

This reduces bandwidth consumption by orders of magnitude while maintaining the robustness of state-based models.

# Metadata Management

CRDTs track concurrency using metadata (Vector Clocks, dots, tombstones). Without management, this grows indefinitely.

- Dotted Version Vectors (DVV): Optimized vector clocks that track individual events to solve the "Actor Explosion" problem.
- Causal Stability: The only safe way to delete a tombstone is to prove that no future message will ever depend on it.



# Industry Adoption

---



## Databases

Riak KV: Pioneering use of CvRDTs (Maps, Sets) with DVV.

Redis Enterprise: Implements "Active-Active" geo-replication using Op-based CRDTs.



## Collaboration

Yjs & Automerge: JS libraries for Google Docs-like apps. Use complex Sequence CRDTs (YATA, RGA) optimized for JSON.



## Local-First

Ink & Switch: CRDTs enable software that keeps data local and syncs peer-to-peer, guaranteeing ownership and offline capability.